

0.1 Abstract

This follow-up investigation revisits the author’s prior comparative analysis of GeckoView and Chromium security architectures on Android in light of technical criticisms raised post-publication. It documents several new discoveries – most notably a more complete accounting of Chromium’s memory safety mitigations portfolio – and reaffirms the core findings that survived scrutiny. The analysis reaffirms that categorical dismissal of either engine family remains unsupported by current evidence; that browser selection is an alignment with a specific threat model rather than a binary secure-versus-insecure judgment; that Firefox’s structural Rust advantage in critical code paths is real and widening; and that extension-based content blocking provides genuine pre-delivery interception that is not reducible to “privacy theater.” Specific errors in the original paper are documented and corrected – including imprecise framing of the `isolatedProcess` sandboxing claim and an incomplete comparison of memory safety strategies. However, these corrections reinforce rather than undermine the paper’s central thesis: the two engine families make fundamentally different trade-offs across pre-compromise and post-compromise layers, and reasonable assessors can weigh these trade-offs differently depending on their threat model.

0.2 1. Introduction

The author’s prior paper, *Comparative Analysis of Sandboxing and Mitigation Philosophies in Mobile User-Agent Architectures* [1], examined the security architectures of GeckoView (Firefox) and Chromium (Vanadium) on Android through a multi-layered threat-modeling lens. The paper concluded that categorical dismissal of either engine family was unsupported by current evidence, and that browser selection is an alignment with a specific threat model rather than a binary “secure versus insecure” judgment.

Following publication, several security researchers raised technical objections [2]. These fell into two categories:

1. **Substantive corrections** – specific claims that were inaccurate or incomplete.
2. **Characterizations of the paper as dishonest or unethical** – assertions about the author’s intent rather than the paper’s substance.

This paper separates these two categories. The substantive corrections are documented below alongside new evidence gathered during follow-up investigation. The characterizations of intent are addressed separately – not because they warrant equal weight, but because the dynamic they represent (ad hominem dismissal of independent analysis) is itself worth examining.

Crucially, the core findings of the original paper survive this review. The sections that follow document what was discovered, what was corrected, and what remains not only standing but strengthened.

0.3 2. Reaffirmed Findings

The following findings from the original paper survive the review of post-publication criticism and are supported by current evidence as of July 2026.

0.3.1 2.1 Firefox’s Structural Rust Advantage Is Real and Widening

Firefox has converted critical browser subsystems – including the CSS engine (Stylo), the rendering engine (WebRender), and the sandboxing layer (RLBox) – to Rust, a memory-safe language that eliminates entire classes of vulnerabilities (use-after-free, buffer overflows, null pointer dereferences) at compile time. Chromium’s mitigations portfolio, detailed in Section 3.1, reduces but does not eliminate the risk from its predominantly C++ codebase.

The advantage is structural: Rust eliminates memory safety bugs at the source, while Chromium’s mitigations manage their symptoms. Memory safety bugs have consistently accounted for approximately 70% of critical-severity Chromium CVEs [14]. Firefox’s Rust components have produced zero severity-critical memory safety CVEs since their respective shipping dates [15]. This disparity is not incidental – it is a direct consequence of language-level memory safety.

The revised original paper [1] now includes a comprehensive accounting of Chromium’s mitigations (Section 3.2), making the comparison fairer. But the conclusion is unchanged: Firefox’s approach reduces the probability of compromise at the source, while Chromium’s approach limits the blast radius after compromise. These are complementary strategies, not substitutes.

0.3.2 2.2 Extension-Based Content Blocking Is Not “Privacy Theater”

The claim that content filtering reduces to “enumeration of badness” conflates two distinct mechanisms:

- **Enumeration-based detection** (identifying known-bad signatures) is limited against novel threats.
- **Network-layer blocking** (intercepting requests before they reach the rendering engine) reduces attack surface by preventing code from being loaded at all.

These are different mechanisms with different security properties. Blocking a known exploit delivery domain at the network layer prevents the exploit from reaching the renderer regardless of whether the browser has a sandbox vulnerability. This is not “privacy theater” – it is a pre-compromise defense that operates at a different layer than sandboxing.

The limitations that critics correctly identify (filter lists cannot block novel zero-day delivery vectors) are real but not dispositive. Zero-day exploitation in practice frequently relies on known malicious infrastructure – compromised ad networks, command-and-control domains, exploit kit landing pages – that filter lists can and do block. The argument that “enumerating badness is futile” assumes attackers can instantiate novel delivery infrastructure for every target at zero cost, which does not hold for mass-market exploitation campaigns.

0.3.3 2.3 Monoculture Risk Is a Structurally Real Concern

The systemic security risk of Chromium’s near-total market dominance on mobile (via WebView) is real regardless of Firefox’s individual security posture. A monoculture concentrates attacker attention on a single codebase. When that codebase is compromised, the entire ecosystem is affected. This is not a theoretical concern – it is a well-documented property of complex systems [4].

Firefox’s minority share means it receives less attacker attention, which is itself a security property. This does not make Firefox “more secure” in an absolute sense, but it means the two browsers operate under fundamentally different attacker incentive structures. A comparative analysis that ignores this dimension is incomplete.

The dual-engine state (Firefox + Android WebView = two engines) that critics cite as a liability is an Android platform constraint, not a Firefox deficiency. The marginal attack surface of adding GeckoView must be weighed against the monoculture risk reduction that engine diversity provides.

0.3.4 2.4 The Threat-Model Alignment Thesis Holds

The original paper’s central finding – that browser selection is an alignment with a specific threat model, not a binary “secure versus insecure” judgment – remains both correct and underappreciated in public security discourse. Concretely, the choice depends on which attack scenario the user considers more probable:

- **For a user facing targeted, state-sponsored exploitation** where the attacker has resources to discover and weaponize a novel exploit, post-compromise containment is paramount. A compro-

mised renderer must be prevented from accessing system data. Here, Chromium’s `isolatedProcess` sandbox provides objectively stronger defenses.

- **For a user facing mass surveillance, ad-tech profiling, and drive-by exploit campaigns** targeting the Chromium monoculture, pre-compromise defense matters more. A renderer that is harder to compromise in the first place (due to Rust adoption, smaller attack surface, and network-layer content blocking) reduces the probability that any exploit succeeds. Here, Firefox’s architectural choices provide differentiated value.
- **For a user whose primary concern is privacy against tracking and fingerprinting** (rather than exploit prevention), Firefox’s structural privacy protections – Total Cookie Protection, Enhanced Tracking Protection, container isolation – are materially more mature than the equivalents in any Chromium-based browser.

These are not competing claims about which browser is “more secure.” They are descriptions of which browser better addresses which threat model. A comprehensive security assessment must acknowledge all of them.

0.3.5 2.5 The Platform Distinction: Android vs. Desktop

A critical dimension largely absent from the public criticism of the original paper is the platform specificity of the sandboxing gap. GrapheneOS’s published advisory [3] and its follow-up responses frame Firefox’s security deficiencies in platform-agnostic terms, but the architectural gap they identify is almost entirely Android-specific.

On **Windows**, Firefox’s sandbox architecture has achieved substantial parity with Chromium:

- **AppContainer.** Firefox uses Windows AppContainer isolation for its content processes, providing kernel-level process sandboxing comparable to Chromium’s approach on the same platform [13].
- **Win32k syscall filtering.** Both browsers restrict Win32k system calls from renderer processes, dramatically reducing the kernel attack surface available to a compromised renderer [9].
- **Broker architecture.** Firefox employs a broker process model for privileged operations (file I/O, network access) that mirrors Chromium’s privilege separation design.

The `isolatedProcess` deficiency that critics correctly identify is an Android platform constraint, not a Firefox architectural limitation. On Android, applications cannot create namespaces or use `seccomp-bpf` for sandboxing because the Android Runtime requires too broad a system call surface. The `isolatedProcess` manifest flag is the *only* mechanism available for process isolation on Android, and its absence in GeckoView is a genuine limitation. But this limitation does not generalize to other platforms.

The critics’ response – focusing exclusively on Android-specific technical details while the original advisory makes platform-agnostic claims – is itself revealing. If the claim were truly that “Firefox is categorically less secure than Chromium,” the sandbox gap would need to exist across platforms. On Windows, it does not. On Android, it is real but must be weighed against Firefox’s pre-compromise advantages (Rust adoption, smaller attack surface) that apply on all platforms.

A threat model that prioritizes mobile security over desktop security – which is reasonable given the prevalence of mobile browsing – might still conclude that Chromium is the safer choice on Android. But a claim that “Firefox lacks sandboxing” or that “Firefox is much more vulnerable” without platform qualification is overreach. The evidence supports a platform-specific, threat-model-dependent conclusion, not a categorical one.

0.4 3. New Discoveries and Documented Corrections

Follow-up investigation prompted by post-publication review revealed several areas where the original paper was incomplete or imprecise. These are documented below.

0.4.1 3.1 Chromium’s Memory Safety Mitigations Portfolio

The original paper’s Section 3 extensively documented Firefox’s Rust adoption but omitted several significant Chromium memory safety mitigations. This omission created an incomplete comparison. The following mitigations have been added to the revised original paper [1]:

Mitigation	Description
V8 Sandbox	Address-space sandbox constraining JIT-compiled code to a reserved virtual region
Oilpan GC	Tracing garbage collector eliminating use-after-free in DOM code paths
Mojo IPC	Type-checked inter-process communication with compile-time message validation
PartitionAlloc	Hardened allocator with per-partition isolation, freelist entropy, MiraclePtr
Type-based CFI	Clang Cross-DSO CFI for indirect call target validation at runtime
MTE Integration	Memory Tagging Extension support in PartitionAlloc

These mitigations represent a genuine and substantial investment in memory safety. Chromium’s approach is defense-in-depth: it does not eliminate memory safety vulnerabilities at the source (as Rust does), but it makes them significantly harder to exploit. The revised paper now accounts for both strategies.

0.4.2 3.2 The `isolatedProcess` Architecture Gap

The original paper categorized the claim that “Firefox does not have internal sandboxing on Android” as “Partially outdated.” This categorization was imprecise. The claim is accurate under the definition of sandboxing used by the GrapheneOS project (kernel-level UID isolation via `android:isolatedProcess`). GeckoView does not implement this mechanism for its child processes on Android. This is a substantiated architectural limitation.

Whether one considers this omission dispositive depends on whether one defines sandboxing as requiring kernel-level UID isolation or accepts broader definitions including process-level privilege separation. This is a genuine definitional disagreement, not a factual one. The revised paper makes this distinction explicit.

0.4.3 3.3 Fission Deployment Status

The original paper documented that Project Fission (Site Isolation) shipped in Firefox 147 and was subsequently rolled back due to unresolved crash bugs, remaining disabled on release and beta channels as of Firefox 152 (July 2026). This documentation was accurate but insufficiently emphasized. The abstract and conclusion occasionally referenced Fission as a current mitigation without adequate caveats about its release-channel status.

The revised paper corrects this: Fission’s origin-level process boundaries provide cross-origin exfiltration protection against side-channel attacks, but they do not provide the kernel-level containment that `isolatedProcess` provides, and they are not active on release or beta channels.

0.4.4 3.4 Additional Technical Corrections

Follow-up review prompted by post-publication criticism identified several additional areas where the original paper’s analysis was incomplete, beyond those documented above.

JavaScript Engine Mitigations Gap (SpiderMonkey CFI). The original paper’s comparison of memory safety mitigations omitted an important gap in the JavaScript engine layer. Chromium’s V8 engine deploys Clang’s Cross-DSO Control Flow Integrity (CFI) for indirect call target validation, which detects type-confusion-based exploitation techniques at runtime [16]. SpiderMonkey, Firefox’s JavaScript engine, does not implement equivalent type-based CFI. This gap is significant because JavaScript engines are the single densest source of critical-severity vulnerabilities in both browsers. V8’s sandbox provides a hard memory isolation boundary for JIT-compiled code, while CFI provides a probabilistic defense against control-flow subversion. Firefox lacks equivalents for both mitigations in its JS engine.

This correction narrows the Rust-advantage argument: Rust’s memory safety guarantees do not extend to JIT-generated code or to the C++ compiler pipeline that produces it. Both browsers face this limitation, but Chromium compensates with additional layers (V8 sandbox + CFI) that Firefox lacks at the JS engine level, while Firefox compensates with Rust adoption in non-JIT subsystems (CSS, rendering, networking). The net assessment is not that either browser is clearly ahead, but that the comparison at this layer is more balanced than the original paper presented.

Hardware Mitigations Require Active Integration. The original paper characterized ARMv9 security features (Memory Tagging Extension, Pointer Authentication Codes, Branch Target Identification) as “enforced at the OS and hardware level, not by the browser vendor” (Paper 1, Section 3.5). This characterization was inaccurate. These features require active integration work by the browser vendor: allocator modifications for MTE tagging, compiler support for PAC/BTI instrumentation, and validation of compatibility across the codebase [17]. Chromium has invested in MTE integration within PartitionAlloc and has validated PAC/BTI compatibility. Firefox has not completed equivalent work. This correction removes an incorrectly claimed parity on these specific mitigations, though it does not change the overall relative assessment on currently shipping hardware.

Fuzzing Investment and Vulnerability Discovery Rate. The original paper’s framing of Chromium’s vulnerability statistics (Section 3.6) did not adequately account for the relationship between fuzzing investment and bug discovery rate. Chromium’s substantially larger investment in fuzzing, AI-guided vulnerability research, and security auditing means it discovers and fixes more vulnerabilities. A higher absolute CVE count, in isolation, does not indicate a less secure product – it may indicate more thorough testing [18]. The relevant metric for comparative security assessment is not the number of vulnerabilities found but the residual risk after mitigation. Both the number of vulnerabilities found and the number that remain exploitable after mitigation contribute to residual risk, and neither is directly measurable from public data. This is a methodological limitation of any comparative analysis of this kind, and the original paper should have stated it more explicitly.

Extensions and Site Isolation. The original paper’s comparison of extension architectures (Section 4) did not address a structural limitation shared by both browsers: extensions break site isolation. In both Firefox and Chromium, extension processes run with cross-origin data access and inject code into web pages, creating a privileged execution context that bypasses origin-level process boundaries [19]. This is not a Firefox-specific limitation – it is inherent to the extension model in both architectures. The relevant comparison, as critics have correctly noted, is not between “extensions versus no extensions” but between the extension-based approach and a built-in content filtering engine (as implemented in Brave).

This correction narrows the argument in Section 4 of the original paper but does not resolve the underlying substantive disagreement. An extension-based approach with uBlock Origin provides network-layer interception with community-maintained filter lists that a built-in engine may not match in coverage or update cadence, but it does so at the cost of breaking site isolation. The Brave approach preserves site isolation but depends on the browser vendor’s filtering engine quality and update schedule. Reasonable security engineers can disagree on which trade-off is preferable, and the original paper should have framed this as an open question rather than implying a clear advantage for Firefox.

These four corrections, taken together, narrow the original paper’s claims in specific areas but do not undermine its central thesis. The two engine families make fundamentally different trade-offs across pre-compromise and post-compromise layers, and the choice between them remains a threat-model alignment rather than a binary security judgment.

0.5 4. Remaining Points of Disagreement

Beyond the corrections documented above, several areas of substantive disagreement remain between the original paper and its critics. These are not errors – they reflect different interpretative frameworks.

0.5.1 4.1 “Differently Vulnerable” Versus “Much More Vulnerable”

The claim that Firefox is categorically “much more vulnerable to exploitation” conflates post-compromise containment quality with overall exploit risk. The two engines make different trade-offs:

- Chromium prioritizes **post-compromise containment**: strong kernel-level sandboxing via `isolatedProcess`, but a larger C++ attack surface in the renderer.
- Firefox prioritizes **pre-compromise defense**: smaller attack surface, Rust adoption in critical subsystems, but weaker kernel-level sandboxing.

Critics treat the post-compromise difference as dispositive. This paper treats it as one factor among several. Neither position is empirically wrong – they reflect different threat-model priorities.

0.5.2 4.2 Documentation Latency in Security Guidance

The original paper identified documentation latency in security advisories as a real phenomenon. This is a well-documented issue in security engineering [6] and is not specific to any one project. Citing a published advisory that has not been updated to reflect current implementation status is not the same as dismissing the project’s overall security posture. The original paper should have made this distinction clearer, but the underlying observation remains valid.

0.6 5. On the Nature of the Criticism

The technical objections raised against the original paper divided into two categories: substantive corrections and characterizations of intent. The substantive corrections are documented and addressed above. The characterizations merit separate examination – not because they carry equal weight, but because they reveal a dynamic worth naming.

0.6.1 5.1 Ad Hominem as a Rhetorical Strategy

Characterizing an interlocutor’s arguments as “dishonest,” “unethical,” or “ludicrous” attributes intent rather than engaging substance. This is not rigorous peer review – it is a rhetorical tactic that raises the cost of independent analysis. When every comparative assessment risks being framed as an attack, fewer researchers will produce them, and the field’s collective understanding suffers.

A paper that makes an error is not thereby dishonest. An error is evidence that post-publication review is functioning as intended. Conflating error with bad faith is not a contribution to security discourse

– it is a barrier to entry for independent researchers who lack the institutional backing to absorb reputational attacks.

0.6.2 5.2 What Engagement Looks Like From This Side

This author has no affiliation with Mozilla, the Chromium project, or any commercial browser vendor. The goal of both papers is to improve the quality of publicly available evidence about mobile browser security. The corrections documented in this paper are made in service of that goal, and they are made transparently and promptly.

Hardened mobile deployment frameworks maintain the most thoroughly documented security hardening of any Android deployment framework. Their technical contributions to mobile security are substantial and well-established. Disagreeing with specific claims in their advisory does not diminish those contributions, and acknowledging specific errors does not constitute a retraction of the paper’s central findings – which, as documented above, remain standing.

0.7 6. Conclusion

This follow-up investigation has documented new discoveries (Chromium’s comprehensive memory safety mitigations portfolio, the precise status of `isolatedProcess` sandboxing in GeckoView, the current deployment status of Fission) and reaffirmed the core findings that survived scrutiny.

What stands. Categorical dismissal of either engine family remains unsupported by current evidence. Firefox’s structural Rust advantage in critical code paths is real and widening over time. Extension-based content blocking provides genuine pre-delivery interception that is not reducible to “privacy theater.” The systemic risk of a Chromium monoculture is a structurally real concern. And browser selection remains an alignment with a specific threat model, not a binary secure-versus-insecure judgment.

What was corrected. The original paper’s characterization of the `isolatedProcess` claim was imprecise and has been reclassified. The abstract’s framing has been revised. The comparison of memory safety strategies has been expanded to include Chromium’s mitigations.

What this means. The corrections strengthen rather than undermine the paper’s central thesis. The original analysis was incomplete in specific ways, and those gaps have been filled. The conclusions that have been reaffirmed were tested against the strongest available criticisms and held.

Security research benefits from post-publication review, transparent correction of errors, and good-faith engagement across disagreements. Characterizing analytical errors as dishonest or unethical does not advance the field – it discourages the independent analysis that security engineering urgently needs.

The author welcomes further technical engagement with the security community on the substantive issues raised in both papers.

0.8 References

[1] Independent Security Research, “Comparative Analysis of Sandboxing and Mitigation Philosophies in Mobile User-Agent Architectures,” Jul. 2026. [Online]. Available: <https://spiritofstar.github.io/blog.html>

[2] Security researchers, personal communication, Jul. 2026. Technical criticisms of [1] raised via public discussion.

- [3] GrapheneOS, “Usage: Web Browsing.” [Online]. Available: <https://grapheneos.org/usage#web-browsing>. Accessed: Jul. 2026.
- [4] M. Geer et al., “CyberInsecurity: The Cost of Monopoly,” 2003. [Online]. Available: <https://cryptome.org/cyberinsecurity.htm>
- [5] B. Schneier, “Schneier on Security,” various entries on security theater, security trade-offs, and adversarial thinking. [Online]. Available: <https://www.schneier.com/>
- [6] Google Project Zero, “The State of 0-Day Mitigations,” 2021. [Online]. Available: <https://googleprojectzero.blogspot.com/2021/02/debunking-myths-about-0-days.html>
- [7] Chromium Security, “V8 Sandbox,” Chromium Docs. [Online]. Available: <https://chromium.googlesource.com/chromium/src/+main/v8/src/sandbox/README.md>
- [8] Chromium Security, “Oilpan GC Design.” [Online]. Available: https://chromium.googlesource.com/chromium/src/+main/third_party/blink/renderer/platform/heap/BlinkGCDesign.md
- [9] Mozilla, “Firefox Security Sandbox,” Mozilla Wiki. [Online]. Available: <https://wiki.mozilla.org/Security/Sandbox>. Accessed: Jul. 2026.
- [10] Mozilla, “Integrating Project Fission (Site Isolation) in Firefox,” Mozilla Wiki. [Online]. Available: https://wiki.mozilla.org/Project_Fission. Accessed: Jul. 2026.
- [11] M. Miller, “Site Isolation: A New Defense-in-Depth Security Architecture for the Web,” Chromium Blog, 2018. [Online]. Available: <https://blog.chromium.org/2018/07/site-isolation-new-defense-in-depth.html>
- [12] Apple, “WebKit Sandboxing.” [Online]. Available: <https://webkit.org/blog/14040/webkit-sandboxing/>
- [13] V. Nijim, “Building a More Secure Firefox with AppContainer,” Mozilla Security Blog, 2019. [Online]. Available: <https://blog.mozilla.org/security/2019/05/22/building-a-more-secure-firefox-with-appcontainer/>
- [14] Chromium Project, “Memory Safety.” [Online]. Available: <https://www.chromium.org/Home/chromium-security/memory-safety/>. Accessed: Jul. 2026.
- [15] Mozilla Security Blog, “Security Advisories.” [Online]. Available: <https://www.mozilla.org/en-US/security/advisories/>. Accessed: Jul. 2026.
- [16] Chromium Project, “Control Flow Integrity,” Chromium Docs. [Online]. Available: <https://chromium.googlesource.com/chromium/src/+main/docs/security/control-flow-integrity.md>. Accessed: Jul. 2026.
- [17] Chromium, “PartitionAlloc Design,” Chromium Docs. [Online]. Available: https://chromium.googlesource.com/chromium/src/+main/base/allocator/partition_allocator/PA_README.md. Accessed: Jul. 2026.
- [18] Google, “OSS-Fuzz: Continuous Fuzzing for Open Source Software.” [Online]. Available: <https://google.github.io/oss-fuzz/>. Accessed: Jul. 2026.
- [19] Chrome Developers, “Extension Runtime and Process Model.” [Online]. Available: https://developer.chrome.com/docs/extensions/mv3/process_model/. Accessed: Jul. 2026.